Informe de operacion - P10

Fecha: 14 de octubre de 2026. Cluster: p9-202300644. Namespace: sa-p9. Las horas de la tabla son de Guatemala (UTC-6). El JSON conserva las marcas UTC.

1. SLO declarados

Disponibilidad: 99% de solicitudes sin HTTP 5xx durante una hora, con presupuesto de 1%. Latencia: 95% de solicitudes dentro de 500 ms durante una hora, con presupuesto de 5%. Se excluyen /health y /metrics.

Una hora permite evaluar una sesion del laboratorio con su historial real. El 99% deja margen para fallas ocasionales; 500 ms evita una espera molesta en el carrito, permitiendo 5% de solicitudes lentas. El indicador de latencia es la proporcion dentro del umbral; el p95 es la metrica adicional del canary. No se afirma un cumplimiento mensual.

2. Fallo inducido

Se agrego temporalmente P10_DEMO_DELAY_MS=1200 a la candidata mediante GitOps, usando la imagen firmada p10-v1.0.5. Solo /orders/graphql espero 1.2 segundos. No se alteraron /health ni la consulta de catalogo que verificaba el Job de P8: la falla fue lentitud real en una ruta de negocio, detectada con Prometheus. La consulta uso un producto inexistente y devolvio el error GraphQL esperado con HTTP 200; no se presenta como una compra exitosa ni se genero un pago.

El gateway registro 1320 ms en la solicitud correlacionada, mientras catalogo registro 2 ms y orders-ms 29.76 ms. El p95 calculado por el histograma fue 2.425 segundos; es una estimacion por buckets, no el tiempo exacto de esa solicitud.

3. Tiempo de deteccion

Marca	Hora local
Primera solicitud con demora	14:33:09
Alerta entra en Pending	14:33:45
Se observa alerta Firing	14:34:16
Se identifica la causa	14:34:18
Analisis falla	14:34:21
Se confirma respuesta estable	14:34:22

La primera señal aparecio a los 36.542 segundos (0.609 minutos). La alerta Firing se observo a los 66.816 segundos (1.114 minutos). La fuente es ApiLatencyErrorBudgetBurn: el ritmo de consumo de la candidata fue 20 veces el permitido, no un umbral de latencia instantanea.

4. Recorrido hasta la causa

El observador consulto la alerta Firing, su metrica de presupuesto, la traza en Tempo y los logs correspondientes en Loki. Ese recorrido de consultas automatizadas tomo **2.469 segundos**, desde observar la alerta hasta encontrar el evento demo_delay. No se presenta como un tiempo manual.

Trace ID: a783c66244eae44ec24cd84fa2cd7ceb. Los tres servicios comparten ese identificador. El log del gateway indica delay_ms=1200 y la traza ubica la demora en ese servicio, no en la base de datos.

5. Tiempo de recuperacion

AnalysisRun: api-gateway-f546c89c7-12-2. Resultado: Failed por p95=2.425 s; el analisis de errores HTTP paso con valor 0.

Argo Rollouts aborto automaticamente la candidata y conservo stable (fdd579dcc). Se comprobo HTTP 200 en 316.187 ms. El tiempo desde la primera solicitud lenta hasta esa respuesta fue **73.181 segundos (1.220 minutos)**. No se intervino para provocar el aborto.

Despues se retiro la variable de demora en GitOps y ArgoCD sincronizo el estado deseado. Ese retiro posterior fue una limpieza del experimento; no fue lo que recupero el servicio.

6. Alerta faltante o sobrante

La primera prueba aborto correctamente, pero la alerta global no disparo: el trafico sano de stable diluyo la lentitud. Se agrego el calculo del presupuesto por pod candidato con el mismo objetivo, usando ventanas de 1 y 5 minutos. La segunda prueba confirmo la alerta y el recorrido completo.

Se elimino la alerta instantanea de errores del canary: Argo ya detiene esa promocion. Se conservan alertas con presupuesto, contexto y runbook. La siguiente mejora seria conservar el historial en almacenamiento persistente y separar los errores GraphQL HTTP 200 de los errores de transporte.

Evidencia: ../evidencias/09-incidente.json y ../evidencias/10-reversion.png.